Session 14 (AIML) - Assignment Questions

Name**: Vaishnavi Praveen Rathi**
College**: Zeal Polytechnic, Narhe, Pune**
Branch**: Artificial Intelligence and Machine Learning (AIML)**
Domain**: AIML (Artificial Intelligence and Machine Learning)**

Github Repo Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link : https://github.com/vaishnavi-rathiii/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Assignment14.ipynb

Q1. Array Properties

Output Screenshots:

```
Array:
[[40 69 13 84 47 82]
 [55 40 23 19 90 59]
 [99 77 46 48 23 53]
 [92 72 88 45 89 96]
 [95 16 49 32 23 69]]

Shape: (5, 6)
Size: 30
Data Type: int32
```

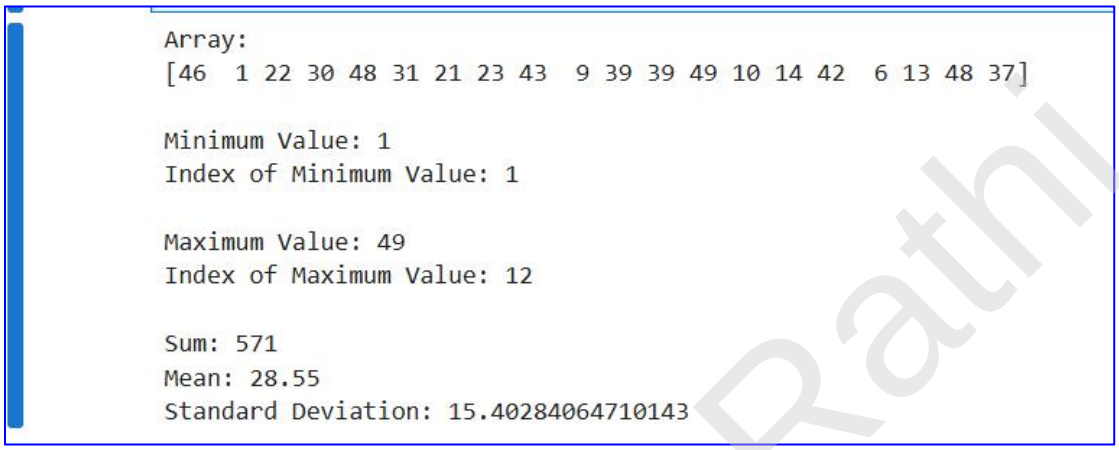
Program Code:

```
""" ASSIGNMENT 14 """

'''
Q1. (Array Properties)

Create a 2D array of shape (5, 6) filled with random integers between 10 and 100.

Print the following:
- Shape of the array
- Total number of elements (size)
- Data type (dtype)
'''

import numpy as np

# Create a 5x6 array with random integers between 10 and 100
arr = np.random.randint(10, 101, (5, 6))

# Display the array
print("Array:")
print(arr)
```

```
# Display array properties
print("\nShape:", arr.shape)
print("Size:", arr.size)
print("Data Type:", arr.dtype)
```

Q2. Statistical Methods - Basic

Output Screenshot:



```
Array:
[46  1 22 30 48 31 21 23 43  9 39 39 49 10 14 42  6 13 48 37]

Minimum Value: 1
Index of Minimum Value: 1

Maximum Value: 49
Index of Maximum Value: 12

Sum: 571
Mean: 28.55
Standard Deviation: 15.40284064710143
```

Program Code:

```
""" ASSIGNMENT 14 """

'''
Q2. (Statistical Methods - Basic)

Generate a 1D array of 20 random numbers between 1 and 50 using
np.random.randint().

Print:
- Minimum value and its index (argmin)
- Maximum value and its index (argmax)
- Sum of all elements
- Mean and Standard Deviation
'''

import numpy as np

# Generate a 1D array of 20 random integers between 1 and 50
arr = np.random.randint(1, 51, 20)

# Display the array
print("Array:")
print(arr)
```

```
# Statistical methods
print("\nMinimum Value:", np.min(arr))
print("Index of Minimum Value:", np.argmin(arr))

print("\nMaximum Value:", np.max(arr))
print("Index of Maximum Value:", np.argmax(arr))

print("\nSum:", np.sum(arr))
print("Mean:", np.mean(arr))
print("Standard Deviation:", np.std(arr))
```

Q3.Statistical Methods on 2D Array

Output Screenshot:

```
Matrix:
[[26 49 40 34 27]
 [46 60 72 61 44]
 [59 39 53 26 26]
 [26 61 75 66 72]]

Minimum Value: 26
Maximum Value: 75

Sum: 962
Mean: 48.1
Standard Deviation: 16.691015547293702

Row-wise Sum:
[176 283 203 300]

Column-wise Sum:
[157 209 240 187 169]
```

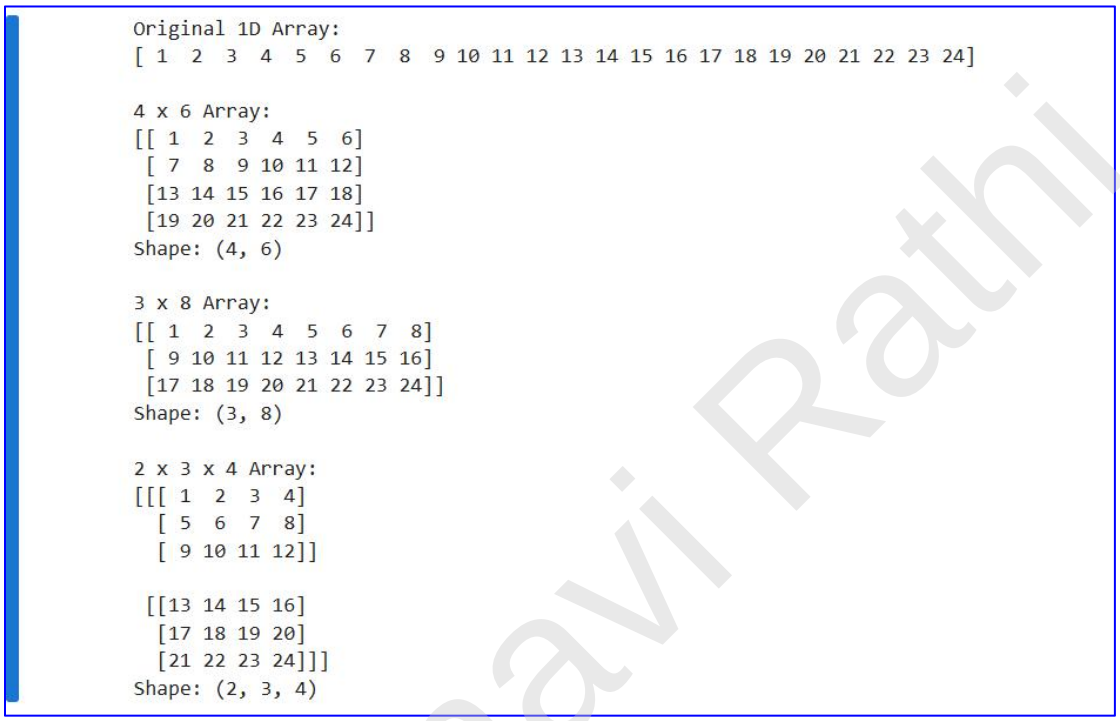
Program Code:

```
""" ASSIGNMENT 14 """

'''
Q3. (Statistical Methods on 2D Array)

Create a 4x5 matrix of random integers between 20 and 80.

For the entire matrix, print:
- Minimum and maximum value
- Sum of all elements
- Mean and Standard Deviation

Then print the sum of each row and each column.
'''

import numpy as np

# Create a 4x5 matrix of random integers between 20 and 80
arr = np.random.randint(20, 81, (4, 5))

# Display the matrix
```

```
print("Matrix:")
print(arr)

# Statistical methods
print("\nMinimum Value:", np.min(arr))
print("Maximum Value:", np.max(arr))

print("\nSum:", np.sum(arr))
print("Mean:", np.mean(arr))
print("Standard Deviation:", np.std(arr))

# Row-wise and column-wise sum
print("\nRow-wise Sum:")
print(np.sum(arr, axis=1))

print("\nColumn-wise Sum:")
print(np.sum(arr, axis=0))
```

Q4. Reshape

Output Screenshot:



```
Original 1D Array:
[ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]

4 x 6 Array:
[[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]]
Shape: (4, 6)

3 x 8 Array:
[[ 1  2  3  4  5  6  7  8]
 [ 9 10 11 12 13 14 15 16]
 [17 18 19 20 21 22 23 24]]
Shape: (3, 8)

2 x 3 x 4 Array:
[[[ 1  2  3  4]
  [ 5  6  7  8]
  [ 9 10 11 12]]

 [[13 14 15 16]
  [17 18 19 20]
  [21 22 23 24]]]
Shape: (2, 3, 4)
```

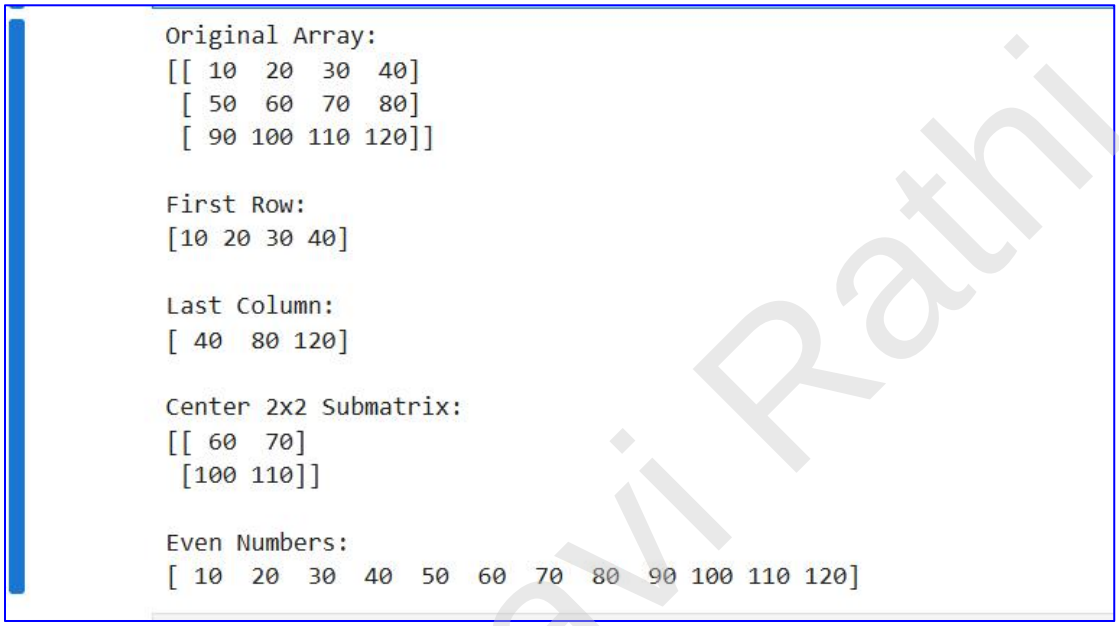
Program Code:

```
""" ASSIGNMENT 14 """

'''
Q4. (Reshape)

Create a 1D array containing numbers from 1 to 24 using np.arange()

- Reshape it into (4, 6)
- Reshape it into (3, 8)
- Reshape it into (2, 3, 4) (3D array)

Print all reshaped arrays with their shapes.
'''

import numpy as np

# Create a 1D array from 1 to 24
arr = np.arange(1, 25)
```

```
# Reshape the array
arr1 = arr.reshape(4, 6)
arr2 = arr.reshape(3, 8)
arr3 = arr.reshape(2, 3, 4)

# Display original array
print("Original 1D Array:")
print(arr)

# Display reshaped arrays
print("\n4 x 6 Array:")
print(arr1)
print("Shape:", arr1.shape)

print("\n3 x 8 Array:")
print(arr2)
print("Shape:", arr2.shape)

print("\n2 x 3 x 4 Array:")
print(arr3)
print("Shape:", arr3.shape)
```

Q5. NumPy Indexing - 1D & 2D

Output Screenshot:



```
Original Array:
[[ 10  20  30  40]
 [ 50  60  70  80]
 [ 90 100 110 120]]

First Row:
[10 20 30 40]

Last Column:
[ 40  80 120]

Center 2x2 Submatrix:
[[ 60  70]
 [100 110]]

Even Numbers:
[ 10  20  30  40  50  60  70  80  90 100 110 120]
```

Program Code:

```
""" ASSIGNMENT 14 """

'''
Q5. (NumPy Indexing - 1D & 2D)

Create the following array:

arr = np.array([[10, 20, 30, 40],
                [50, 60, 70, 80],
                [90, 100, 110, 120]])

Perform the following indexing:
- Extract first row
- Extract last column
- Extract center 2x2 submatrix
- Extract all even numbers from the array using boolean indexing
'''

import numpy as np
```

```
# Create the array
arr = np.array([[10, 20, 30, 40],
                [50, 60, 70, 80],
                [90, 100, 110, 120]])

# Display the array
print("Original Array:")
print(arr)

# Extract first row
print("\nFirst Row:")
print(arr[0])

# Extract last column
print("\nLast Column:")
print(arr[:, 3])

# Extract center 2x2 submatrix
print("\nCenter 2x2 Submatrix:")
print(arr[1:3, 1:3])

# Extract all even numbers using boolean indexing
print("\nEven Numbers:")
print(arr[arr % 2 == 0])
```

Q6. Advanced Indexing

Output Screenshot:

```
Original Array:
[[31 26 62 94 31]
 [ 6 63 63 29 91]
 [84 19 19 16 82]
 [71  3 72  7 21]
 [74 72 96 99 31]]

Diagonal Elements:
[31 63 19  7 31]

Elements Greater Than 50:
[62 94 63 63 91 84 82 71 72 74 72 96 99]

Modified Array:
[[31  0 62 94 31]
 [ 0 63 63  0 91]
 [84  0  0  0 82]
 [71  0 72  0  0]
 [74 72 96 99 31]]
```

Program Code:

```
""" ASSIGNMENT 14 """

'''
Q6. (Advanced Indexing)

Create a 5x5 matrix of random integers between 1 and 100.

Print:
- Diagonal elements
- Elements greater than 50
- Replace all elements less than 30 with 0
- Print the modified array
'''

import numpy as np

# Create a 5x5 matrix of random integers between 1 and 100
arr = np.random.randint(1, 101, (5, 5))

# Display the original array
```

```
print("Original Array:")
print(arr)

# Print diagonal elements
print("\nDiagonal Elements:")
print(np.diag(arr))

# Print elements greater than 50
print("\nElements Greater Than 50:")
print(arr[arr > 50])

# Replace elements less than 30 with 0
arr[arr < 30] = 0

# Display the modified array
print("\nModified Array:")
print(arr)
```

Q7. Arithmetic Operations

Output Screenshot:

```
Addition:
[11 22 33 44]

Subtraction:
[ 9 18 27 36]

Multiplication:
[ 10  40  90 160]

Division:
[10. 10. 10. 10.]

Element-wise Power:
[     10     400   27000 2560000]

Modulo Operation:
[0 0 0 0]
```

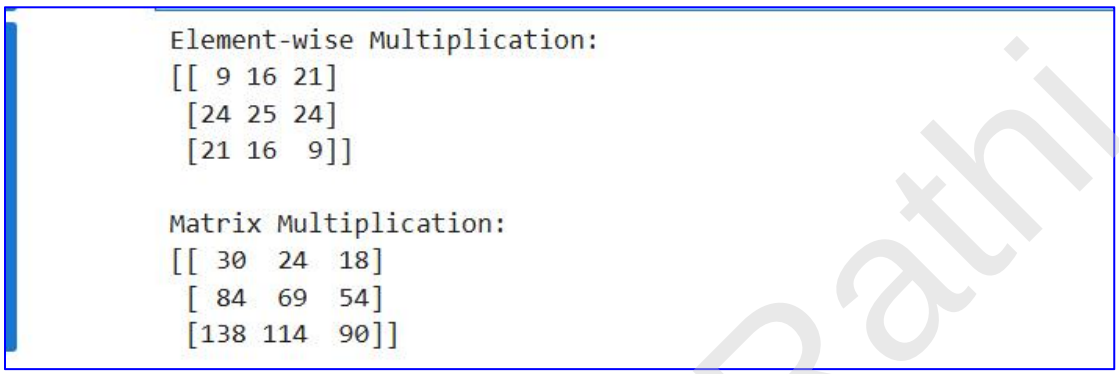
Program Code:

```
""" ASSIGNMENT 14 """

'''
Q7. (Arithmetic Operations)

Create two arrays:

a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])

Perform and print:
- Addition
- Subtraction
- Multiplication
- Division
- Element-wise Power (**)
- Modulo Operation (%)
'''

import numpy as np
```

```
# Create arrays
a = np.array([10, 20, 30, 40])
b = np.array([1, 2, 3, 4])

# Perform arithmetic operations
print("Addition:")
print(a + b)

print("\nSubtraction:")
print(a - b)

print("\nMultiplication:")
print(a * b)

print("\nDivision:")
print(a / b)

print("\nElement-wise Power:")
print(a ** b)

print("\nModulo Operation:")
print(a % b)
```

Q8.Matrix Multiplication

Output Screenshot:



```
Element-wise Multiplication:
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]

Matrix Multiplication:
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]
```

Program Code:

```
""" ASSIGNMENT 14 """

'''
Q8. (Matrix Multiplication)

Create two 3x3 matrices:

A = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])

B = np.array([[9,8,7],
              [6,5,4],
              [3,2,1]])

Perform:
- Element-wise multiplication (*)
- Matrix multiplication (@ or np.dot())

Print both results and explain the difference in comments.
'''

import numpy as np
```

```
# Create matrices
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

B = np.array([[9, 8, 7],
              [6, 5, 4],
              [3, 2, 1]])

# Element-wise multiplication
element_mul = A * B

# Matrix multiplication
matrix_mul = A @ B

# Display results
print("Element-wise Multiplication:")
print(element_mul)

print("\nMatrix Multiplication:")
print(matrix_mul)

# Difference:
# (*) multiplies corresponding elements of both matrices.
# (@) performs matrix multiplication using rows of the first matrix
# and columns of the second matrix.
```

Q9.Combined - Properties + Operations + Indexing

Output Screenshot:

```
Original Matrix:
[[ 0.13566687 -1.53683705  0.77820848  1.01218647 -0.70290862 -0.08531817]
 [ 0.37558123 -0.33600087  0.50987694  0.57040636 -1.26962402 -0.08652411]
 [ 0.57758503  1.12973291  1.0926021   0.71595469 -0.39953793  0.28171931]
 [ 1.07828643 -0.94117101 -1.14940704 -0.14664417 -0.4612903  -0.96169407]
 [-0.35333682  1.62720859  1.8953161  -2.03340434 -0.39958177 -0.64504287]
 [ 0.20798695  0.92174224  0.79521936 -0.58117255 -1.42070192 -1.89857559]]

Shape: (6, 6)
Size: 36
Data Type: float64

Index of Maximum Value: (np.int64(4), np.int64(2))
Index of Minimum Value: (np.int64(4), np.int64(3))

Top-left 3x3 Submatrix:
[[ 0.13566687 -1.53683705  0.77820848]
 [ 0.37558123 -0.33600087  0.50987694]
 [ 0.57758503  1.12973291  1.0926021 ]]

Mean of Modified Matrix: 0.8087237025198126
```

Program Code:

```
""" ASSIGNMENT 14 """

'''
Q9. (Combined - Properties + Operations + Indexing)

Generate a 6x6 matrix of random numbers using np.random.randn().

Print:
- Shape, Size, and Data Type
- Index of the Maximum and Minimum Value
- Top-left 3x3 Submatrix
- Replace all negative numbers with their absolute value
- Mean of the Modified Matrix
'''

import numpy as np

# Generate a 6x6 matrix of random numbers
arr = np.random.randn(6, 6)
```

```
# Display the original matrix
print("Original Matrix:")
print(arr)

# Array properties
print("\nShape:", arr.shape)
print("Size:", arr.size)
print("Data Type:", arr.dtype)

# Index of maximum and minimum values
print("\nIndex of Maximum Value:", np.unravel_index(np.argmax(arr),
arr.shape))
print("Index of Minimum Value:", np.unravel_index(np.argmin(arr), arr.shape))

# Extract top-left 3x3 submatrix
print("\nTop-left 3x3 Submatrix:")
print(arr[:3, :3])

# Replace negative values with absolute values
arr = np.abs(arr)

# Mean of modified matrix
print("\nMean of Modified Matrix:", np.mean(arr))
```

Q10. Mini Project - Student Performance Analysis

Output Screenshot:

```
Student Marks:
[[ 97  77 100  62  88]
 [ 88  85  96  85  98]
 [ 77  73  87  98  52]
 [ 67  73  85  93  42]
 [ 94  46  70  70  79]
 [ 81  62  44  98  90]
 [ 93 100  52  95  38]
 [ 87  71  37  35  43]
 [ 32  79  64  39  70]
 [100  41  72  57  79]]

Total Marks:
[424 452 387 360 359 375 378 273 284 349]

Average Marks:
[84.8 90.4 77.4 72.  71.8 75.  75.6 54.6 56.8 69.8]

Student with Highest Total Marks: 1
Total Marks: 452

Student with Lowest Total Marks: 7
Total Marks: 273

Class Mean: 72.82
Class Standard Deviation: 20.709118764447705

Marks of Top 3 Students:
[[ 77  73  87  98  52]
 [ 97  77 100  62  88]
 [ 88  85  96  85  98]]
```

Program Code:

```
""" ASSIGNMENT 14 """

'''
Q10. (Mini Project - Student Performance Analysis)

Create a NumPy-based Student Marks Analyzer:

- Generate a 2D array of shape (10 students × 5 subjects)
  with random marks between 30 and 100.
- Print the array.
- Calculate total marks and average for each student.
- Find the student with highest and lowest total marks.
```

```
- Calculate overall class mean and standard deviation.
- Demonstrate indexing to extract marks of top 3 students.
'''

import numpy as np

try:
    # Generate marks for 10 students and 5 subjects
    marks = np.random.randint(30, 101, (10, 5))

    # Display marks
    print("Student Marks:")
    print(marks)

    # Calculate total marks and average
    total = np.sum(marks, axis=1)
    average = np.mean(marks, axis=1)

    print("\nTotal Marks:")
    print(total)

    print("\nAverage Marks:")
    print(average)

    # Find highest and lowest scorer
    highest = np.argmax(total)
    lowest = np.argmin(total)

    print("\nStudent with Highest Total Marks:", highest)
    print("Total Marks:", total[highest])

    print("\nStudent with Lowest Total Marks:", lowest)
    print("Total Marks:", total[lowest])

    # Overall class statistics
    print("\nClass Mean:", np.mean(marks))
    print("Class Standard Deviation:", np.std(marks))

    # Extract top 3 students using indexing
    top3 = np.argsort(total)[-3:]

    print("\nMarks of Top 3 Students:")
    print(marks[top3])

except Exception as e:
    print("Error:", e)
```
